

Polimorfismo

polimorfismo

- ❖ O polimorfismo permite que a mesma chamada de método realize ações diferentes dependendo do tipo de objeto.
- ❖ **Depende do binding**
 - Seleção do método a usar numa dada chamada
 - Durante a compilação – static binding / early binding
 - Durante a execução – dynamic binding
- ❖ O polimorfismo é um conceito - não específico de Java
 - Mesma chamada de método, comportamento diferente
- ❖ A ligação dinâmica é uma solução técnica durante a execução
 - O Java usa-o para escolher o método correto em tempo de execução

Static and dynamic binding

❖ Static binding

- Seleciona durante compilação

```
class Test {  
    static void hello() {  
        System.out.println("Hello");  
    }  
}
```

❖ Dynamic binding

- Só consegue selecionar durante a execução

```
class Parent {  
    public void showMessage() {  
        System.out.println("This is a method from the Parent class.");  
    }  
}  
  
// Child class that tries to inherit from Parent  
class Child extends Parent {  
    public void showMessage() {  
        System.out.println("Trying to override a method.");  
    }  
}
```

Type	When decided	Example
Static binding	Compile time	final, static, private methods
Dynamic binding	Runtime	Overridden methods

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Calculator calc = new Calculator();  
  
        System.out.println(calc.add(2, 3));  
        System.out.println(calc.add(2.5, 3.5));  
        System.out.println(calc.add(1, 2, 3));  
    }  
}
```

Methods have the **same name but different parameters**.

The compiler decides **which method to call based on parameters**.

```
class Calculator {  
    int add(int a, int b) { ①  
        return a + b;  
    }  
    double add(double a, double b) { ②  
        return a + b;  
    }  
    int add(int a, int b, int c) { ③  
        return a + b + c;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
        System.out.println(calc.add(2, 3)); ①  
        System.out.println(calc.add(2.5, 3.5)); ②  
        System.out.println(calc.add(1, 2, 3)); ③  
    }  
}
```

Methods have the **same name but different parameters.**

The compiler decides **which method to call based on parameters.**

```
5  
6.0  
6
```

Static binding

The compiler already knows **which method to call**.

```
class Test {  
    static void hello() {  
        System.out.println("Hello");  
    }  
}
```

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void makeSound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void makeSound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Animal a1 = new Dog(); // parent reference → Dog object  
        Animal a2 = new Cat(); // parent reference → Cat object  
  
        a1.makeSound();  
        a2.makeSound();  
    }  
}
```

The **same method call**

`makeSound()`

produces different behavior depending on
the object (Dog or Cat).

Dog barks

Cat meows

Dynamic binding

- ❖ *Mecanismo de seleção do métodos*
 - também referido como *late binding* ou *run-time binding*
 - Todos os métodos **são late binding**.
- ❖ Seleção baseada na forma (i.e., classe) da referência usada na invocação ao objeto.

```
class Parent {  
  
    public void showMessage() {  
        System.out.println("This is a method from the Parent class.");  
    }  
}  
  
// Child class that tries to inherit from Parent  
class Child extends Parent {  
  
    public void showMessage() {  
        System.out.println("Trying to override a method.");  
    }  
}
```


Polimorfismo e interfaces / classes

- ❖ *Possível entre classes diferentes que partilham mesmo interface:*
 - Através da herança - classes bases
 - podem implementar mesmo java interface

o tipo declarado na referência não precisa de ser exatamente o mesmo tipo do objeto para o qual aponta – pode ser de qualquer tipo derivado

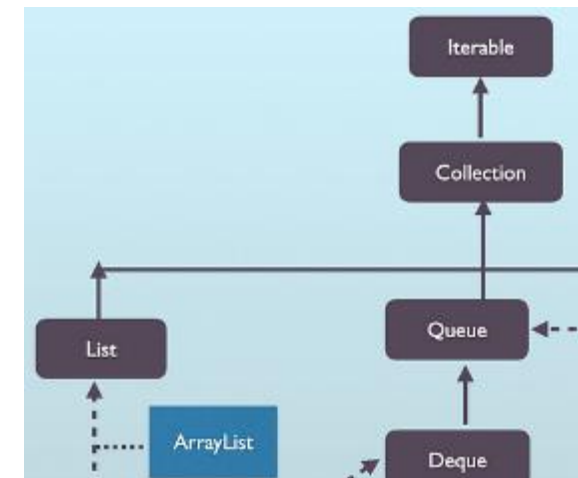
```
Circulo c1 = new Alvo(...);  
Object obj = new Circulo(...);
```

Referência polimórfica

```
T ref1 = new S();  
// OK desde que todo o S seja um T
```

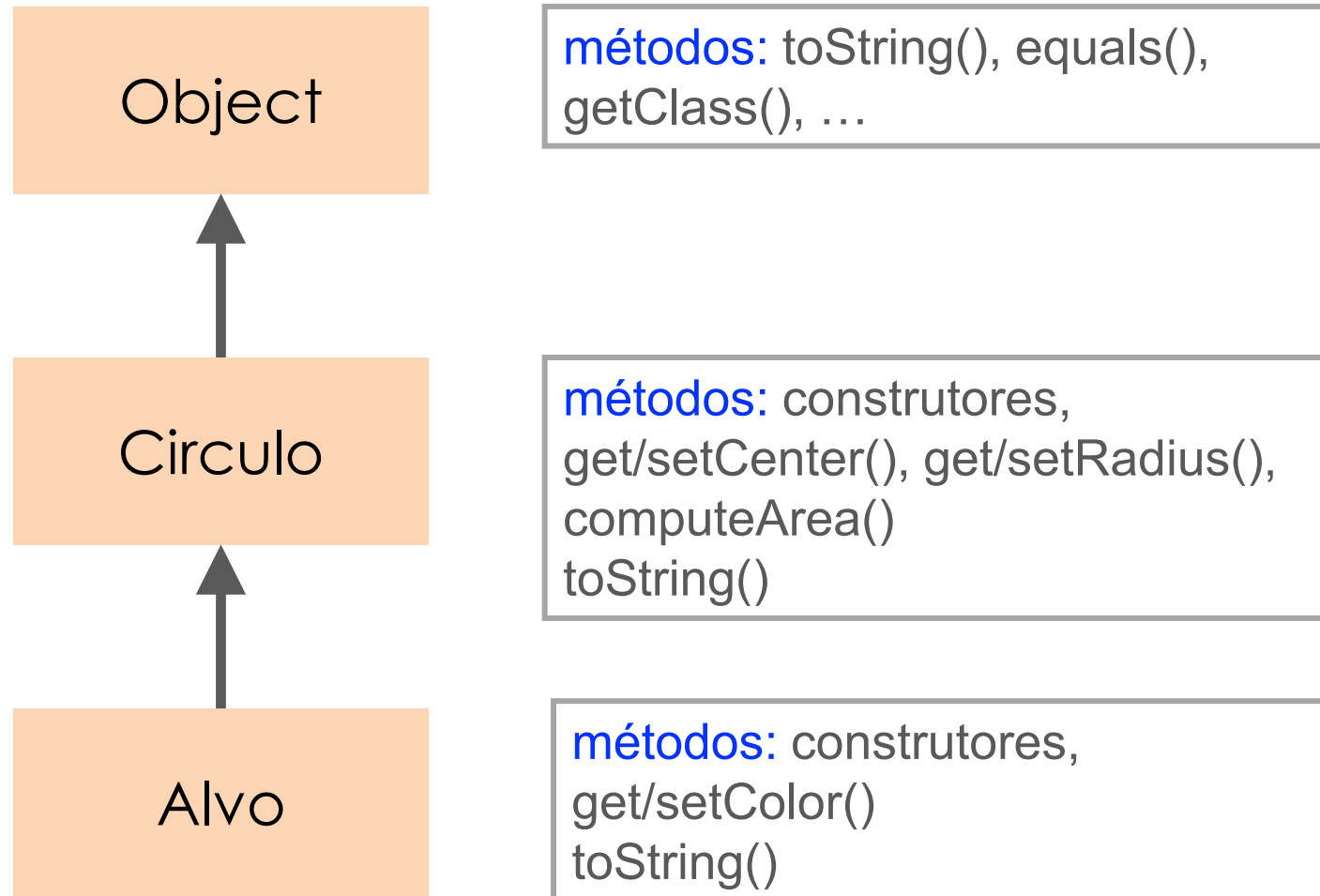
Polimorfismo, herança e interfaces

```
List<Integer> l = new ArrayList<>();  
Collection<Integer> cl = l;  
  
l.add(1);  
cl.add(2);  
  
// Convert Collection to an Integer array  
Integer[] arrayFromCollection = cl.toArray(new Integer[0]);  
Integer[] arrayFromList = l.toArray(new Integer[0]);  
  
// Print the arrays  
System.out.println("Array from Collection: " + Arrays.toString(arrayFromCollection));  
System.out.println("Array from List: " + Arrays.toString(arrayFromList));
```



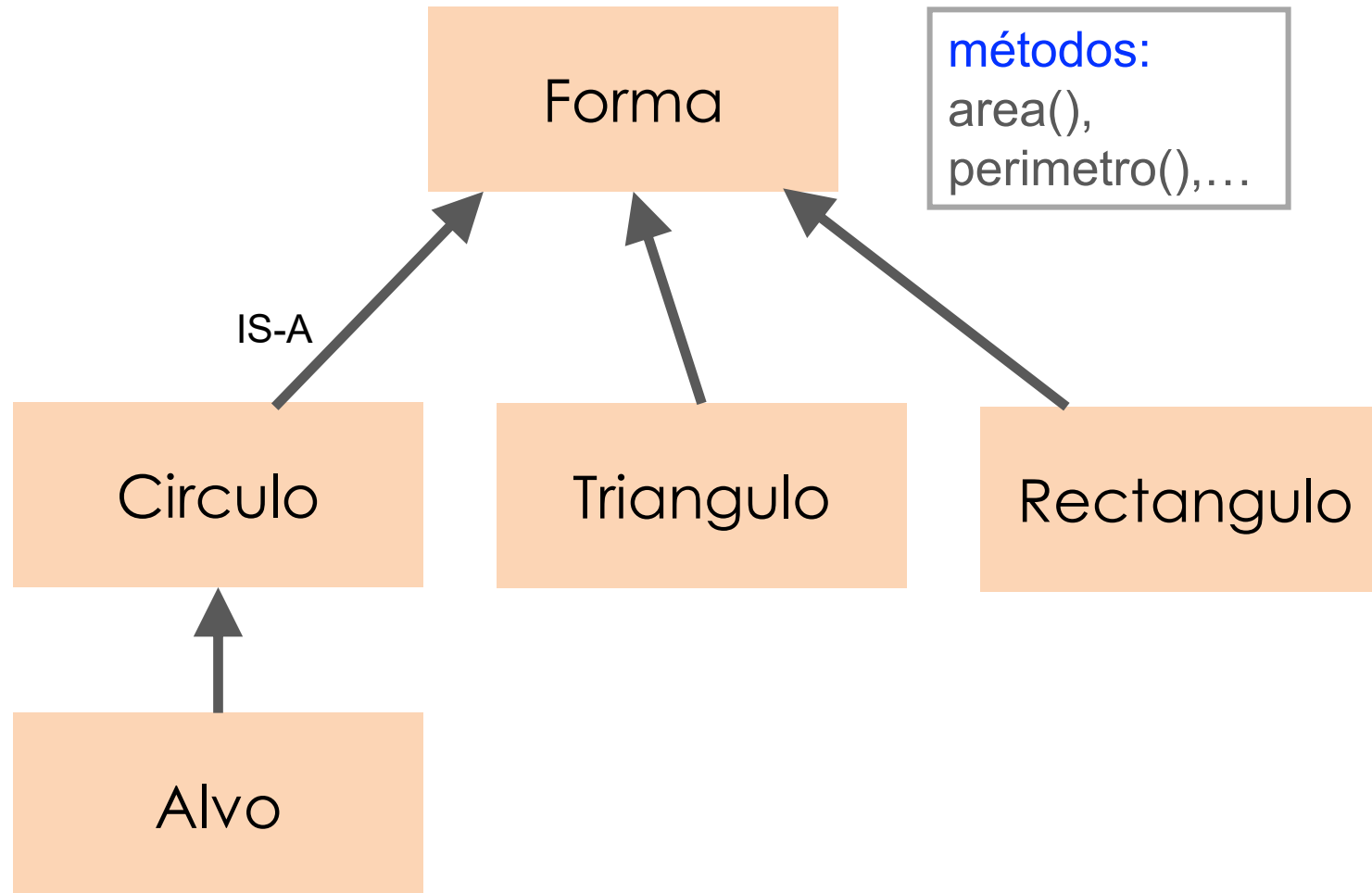
Polimorfismo e herança

Exemplo de herança



Classes derivadas e classes mães não são iguais

Exemplo de herança



Instanceof

- ❖ **Operador que indica se uma referência é membro de uma classe ou interface**
- ❖ Exemplo, considerando

```
class Dog extends Animal implements Pet {...}  
Animal fido = new Dog();
```

- ❖ as instruções seguintes são true:

```
if (fido instanceof Dog) ..  
if (fido instanceof Animal) ..  
if (fido instanceof Pet) ..
```

<https://www.baeldung.com/java-instanceof>

<https://www.programiz.com/java-programming/instanceof>

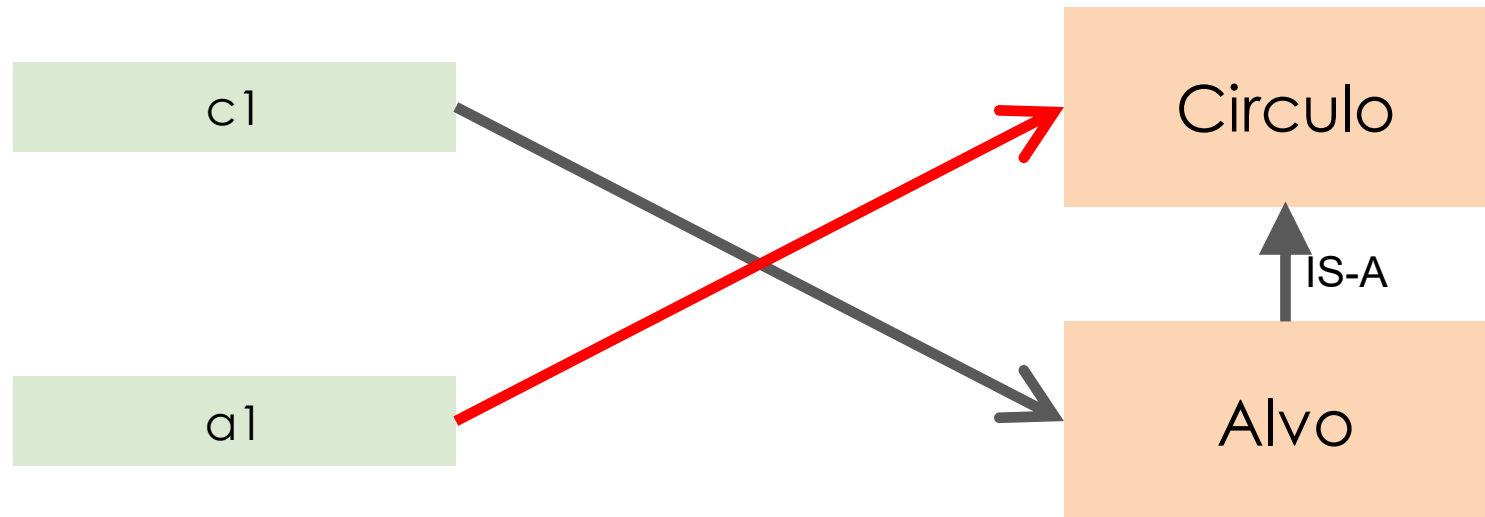
Polimorfismo - exemplos

```
Circulo c1 = new Circulo(1,1,5);  
Alvo a1 = new Alvo(1,1,5, 10);
```



Polimorfismo - exemplos

```
Circulo c1 = new Alvo(1,1,5, 10);  
Alvo a1 = new Circulo(1,1,5); // erro
```



Upcasting e downcasting

- ❖ O tipo do objeto pode ser testado com o operador instanceof

```
Circulo c2 = new Circulo(1.5f, 10, 20);
```

```
fc2 = (Alvo) c2;
```

run-time error:
ClassCast exception

```
if (c3 instanceof Alvo)  
    fc2 = (Alvo) c3;
```

OK

Upcasting e downcasting

```
double z = 2.75;  
int k = (int) z;  
float x = k;  
double w = 5;
```

downcast, $k \leftarrow 2$

upcast automático
 $x \leftarrow 2.0$; $w \leftarrow 5.0$

```
Alvo fc1 = new Alvo(1.5, 10, 20,  
Color.red);
```

OK – um Alvo é um Circulo

```
Circulo c1;  
c1 = fc1;
```

Erro! – c1 é uma referência para Circulo. Mesmo
que aponte para um Alvo precisa de downcast

```
Alvo fc2;  
fc2 = c1;
```

```
fc2 = (Alvo) c1;
```

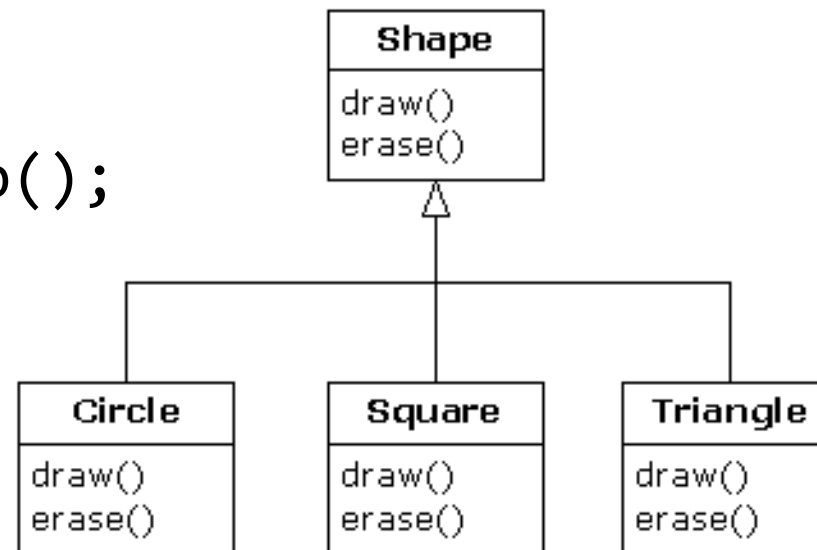
OK

Exemplo 1

```
Shape s = new Shape();  
s.draw();
```

```
Circulo c = new Circulo();  
c.draw();
```

```
Shape s2 = new Circulo();  
s2.draw();
```



Exemplo 2

```
class Shape {  
    void draw() { System.out.println("I am a Shape"); }  
}  
  
class Circle extends Shape {  
    void draw() { System.out.println("I am a Circle"); }  
}  
  
class Square extends Shape {  
    void draw() { System.out.println("I am a Square"); }  
}  
  
public class ShapeSet {  
  
    private static Shape randomShape() {  
        if (Math.random() < 0.5) return new Circle();  
        return new Square();  
    }  
  
    Run | Debug  
    public static void main(String[] args) { args = String[0]@7  
        Shape[] shapes = new Shape[8]; shapes = Shape[8]@8  
        for (int i=0; i<shapes.length; i++)  
            shapes[i] = randomShape();  
  
        for (Shape s: shapes) s = Circle@17, shapes = Shape[8]@8  
        s.draw(); s = Circle@17  
    }  
}
```

I am a Circle
I am a Square
I am a Circle
I am a Square
I am a Circle
I am a Square
I am a Square
I am a Square

Classes abstratas

LEGO Analogy: interface vs Abstract class

❖ interface

- is like the **LEGO standard for connectors**.

- ❖ **doesn't define a brick**, it only defines **what any brick must be able to do**.

```
interface ConnectableBrick {  
    void connectTop();  
    void connectBottom();  
}
```

- ❖ defines **how pieces connect**
- ❖ does **not define the piece itself**

- ❖ An **abstract class** is like a **base LEGO brickdesign**.

- ❖ already has:

- some **built structure**
- but still requires **some parts to be finished**

```
abstract class VehicleBrick {  
  
    void attachWheels() {  
        System.out.println("Wheels attached");  
    }  
  
    abstract void buildTop();  
}
```

- ❖ **partly built brick**

- **defines shared structure**
- **forces subclasses to finish the design**

LEGO Analogy: class

❖ A class is the **real LEGO piece you can use.**

```
class CarBrick extends VehicleBrick implements LegoConnectable {  
    public void connectTop() {  
        System.out.println("Connected on top");  
    }  
  
    public void connectBottom() {  
        System.out.println("Connected on bottom");  
    }  
  
    void buildTop() {  
        System.out.println("Car roof built");  
    }  
}  
This is the actual usable LEGO brick.
```



LEGO Analogy: class

❖ A class is the **real LEGO piece you can use.**

```
class CarBrick extends VehicleBrick implements LegoConnectable {  
  
    public void connectTop() {  
        System.out.println("Connected on top");  
    }  
  
    public void connectBottom() {  
        System.out.println("Connected on bottom");  
    }  
  
    void buildTop() {  
        System.out.println("Car roof built");  
    }  
}
```

This is the **actual usable LEGO brick.**



```
abstract class VehicleBrick {  
  
    void attachWheels() {  
        System.out.println("Wheels attached");  
    }  
  
    abstract void buildTop();  
}
```

Classes Abstratas versus Interfaces

Use an Abstract Class When

You want to:

- share **code between related classes**
- define **common fields**
- provide **partial implementation**

Use an Interface When

You want to define a **capability or behavior** that many unrelated classes can implement.

Feature	Abstract Class	Interface
Instantiation	Cannot instantiate	Cannot instantiate
Inheritance keyword	<code>extends</code>	<code>implements</code>
Multiple inheritance	✗ Only one	✓ Multiple allowed
Fields	Can have instance variables	Only constants (<code>public static final</code>)
Methods	Abstract + concrete	Mostly abstract (plus default/static)
Constructors	✓ Yes	✗ No
State	Can maintain state	Usually stateless

Classes abstratas

- ❖ Incomplete “incompleta”
- ❖ Must contain **Abstract methods** (no body)
- ❖ Provides common structure
 - Common state (fields)
 - Some implemented methods
 - Some methods that subclasses must implement
- ❖ Can contain
 - Abstract methods (no body)
 - Concrete methods (with implementation)
 - Fields
 - Constructors

exemplo

```
abstract class Animal {  
    // concrete method  
    void eat() {  
        ① System.out.println("Animal eats food");  
    }  
  
    // abstract method  
    abstract void makeSound();  
}
```

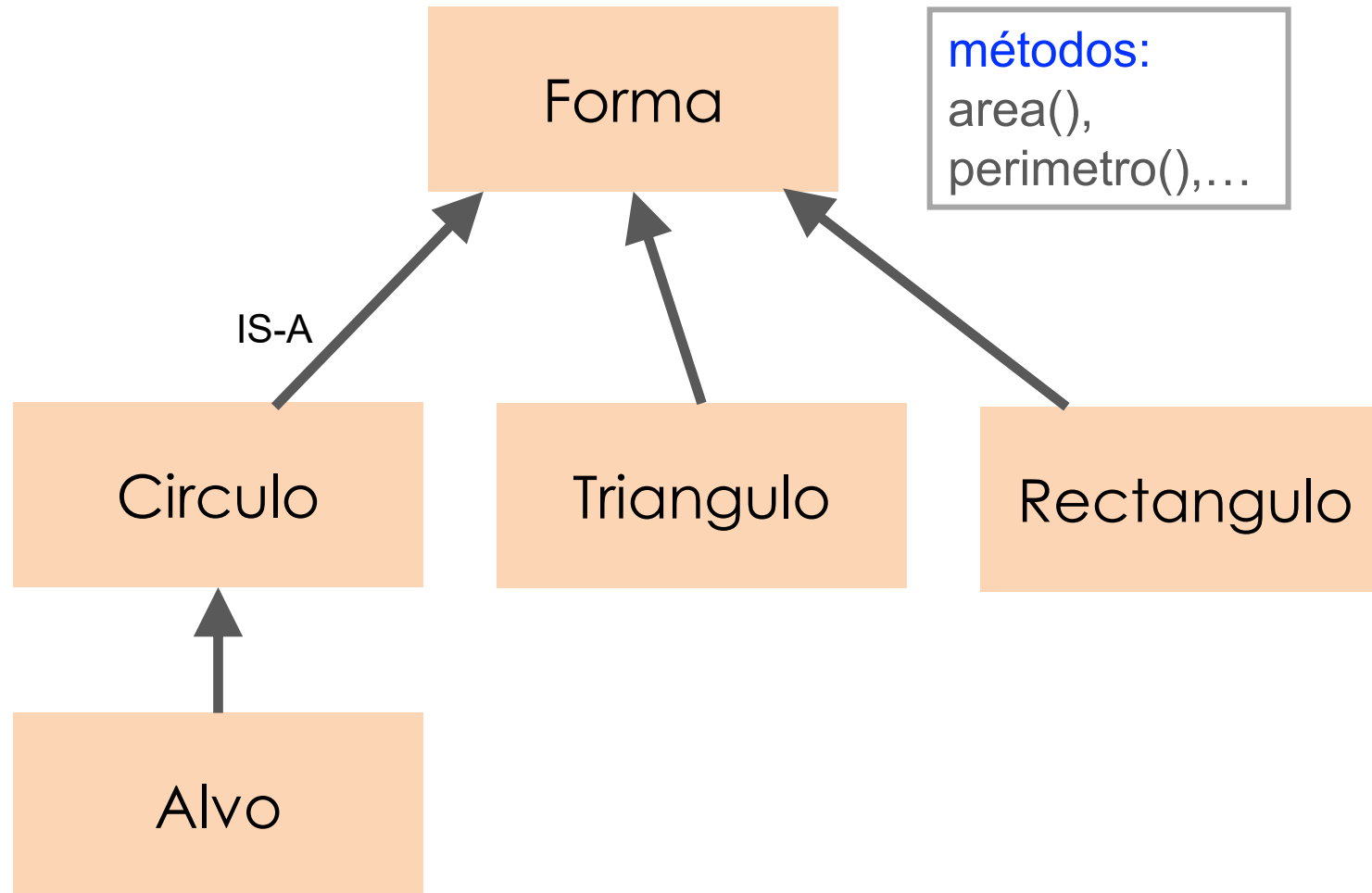
```
class Dog extends Animal {  
    @Override  
    ② void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    ③ void makeSound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal a1 = new Dog();  
        Animal a2 = new Cat();  
  
        a1.eat();  
        a1.makeSound();  
  
        a2.eat();  
        a2.makeSound();  
    }  
}
```

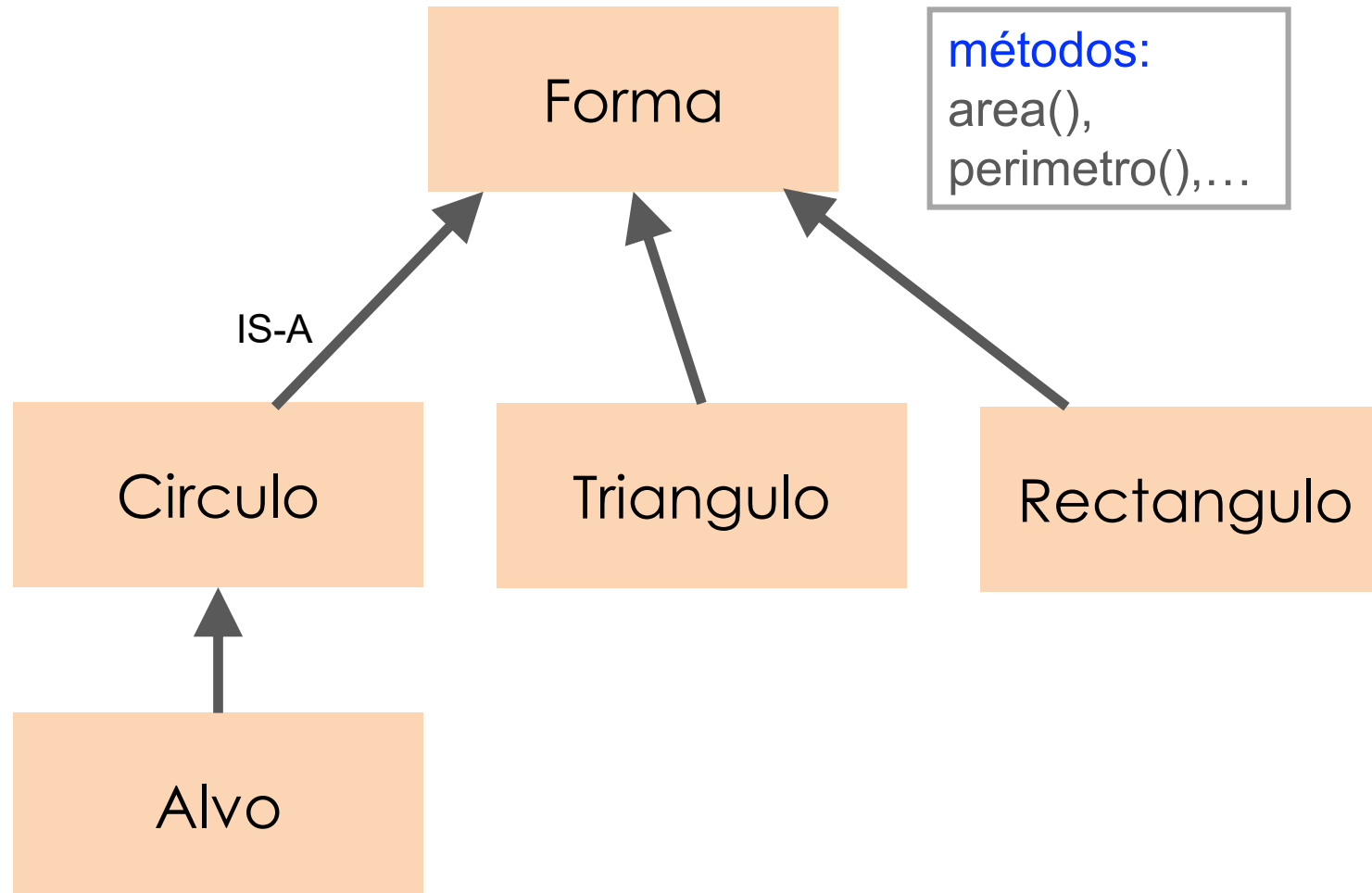
```
① Animal eats food  
Dog barks ②  
① Animal eats food  
Cat meows ③
```

Exemplo de herança



Exemplo de herança

❖ Como implementamos os métodos de Forma?



Classes abstratas

```
public abstract class Forma {  
    // pode definir constantes  
    public static final double DOUBLE_PI = 2*Math.PI;  
  
    // pode declarar métodos abstractos  
    public abstract double area();  
    public abstract double perimetro();  
  
    // pode incluir métodos não abstractos  
    public String aka() { return "euclidean"; }  
}  
Forma f;           // OK. Podemos criar uma variável do tipo Forma  
f = new Forma();   // Erro! Não podemos criar Formas
```

Classes abstratas

```
public abstract class Forma {  
    // pode definir constantes  
    public static final double DOUBLE_PI = 2*Math.PI;  
  
    // pode declarar métodos abstractos  
    public abstract double area();  
    public abstract double perimetro();  
  
    // pode incluir métodos não abstractos  
    public String aka() { return "euclidean"; }  
}  
Forma f;           // OK. Podemos criar uma variável do tipo Forma  
f = new Forma();  // Erro! Não podemos criar Formas
```


Classes abstratas

- ❖ Num processo de herança a classe só deixa de ser abstrata quando implementar todos os métodos abstratos.

```
public class Circulo extends Forma {  
    protected double r;  
  
    public double area() {  
        return Math.PI*r*r;  
    }  
  
    public double perimetro() {  
        return DOUBLE_PI*r;  
    }  
}  
  
Forma f;  
f = new Circulo();    // OK! Podemos criar Circulos
```

Classes abstratas

- ❖ Num processo de herança a classe só deixa de ser abstrata quando implementar todos os métodos abstratos.

```
public class Circulo extends Forma {  
    protected double r;  
  
    public double area() {  
        return Math.PI*r*r;  
    }  
  
    public double perimetro() {  
        return DOUBLE_PI*r;  
    }  
}
```

```
Forma f;  
f = new Circulo();    // OK! Podemos criar Circulos
```

```
public abstract class Forma {  
    // pode definir constantes  
    public static final double DOUBLE_PI = 2*Math.PI;  
  
    // pode declarar métodos abstractos  
    public abstract double area();  
    public abstract double perimetro();  
  
    // pode incluir métodos não abstractos  
    public String aka() { return "euclidean"; }  
}
```

Classes abstratas e Polimorfismo

```
abstract class Figura {
    abstract void doWork();
    protected int cNum;
}

class Circulo extends Figura {
    Circulo(int i) { cNum = i; }
    void doWork() { System.out.println("Circulo"); }
}

class Alvo extends Circulo {
    Alvo(int i) { super(i); }
    void doWork() { System.out.println("Alvo"); }
}

class Quadrado extends Figura {
    void doWork() { System.out.println("Quadrado"); }
}

public class ArrayOfObjects {
    public static void main(String[] args) {

        Figura[] anArray = new Figura[10];
        for (int i = 0; i < anArray.length; i++) {
            switch ((int) (Math.random() * 3)) {
                case 0 : anArray[i] = new Circulo(i); break;
                case 1 : anArray[i] = new Alvo(i); break;
                case 2 : anArray[i] = new Quadrado(); break;
            }
        }
        // invoca o método doWork sobre todas as Figura da tabela
        // -- Polimorfismo
        for (int i = 0; i < anArray.length; i++) {
            System.out.print("Figura(" + i + ") --> ");
            anArray[i].doWork();
        }
    }
}
```

Figura(0) --> Quadrado

Figura(1) --> Circulo

Figura(2) --> Quadrado

Figura(3) --> Circulo

Figura(4) --> Quadrado

Figura(5) --> Alvo

Figura(6) --> Circulo

Figura(7) --> Circulo

Figura(0) --> Circulo

Figura(1) --> Quadrado

Figura(2) --> Alvo

Figura(3) --> Quadrado

Figura(4) --> Alvo

Figura(5) --> Quadrado

Figura(6) --> Quadrado

Figura(7) --> Quadrado

Figura(8) --> Circulo

Figura(9) --> Quadrado

Final ou Como controlar o polimorfismo durante a compilação

Static binding

Final

- ❖ O classificador final indica "não pode ser mudado"
- ❖ A sua utilização pode ser feita sobre:
 - Dados - constantes
 - Métodos - não redefiníveis
 - Classes - não herdadas
 - Fixar referencias a objectos

Usage

`final variable`

`final method`

`final class`

Case

`final variable`

`final method`

`final class`

`final reference`

Allowed

assign once

inherit

create objects

modify object state

Not Allowed

reassign value

override

extend class

change reference

Comparable – polimorfismo na practica com collections

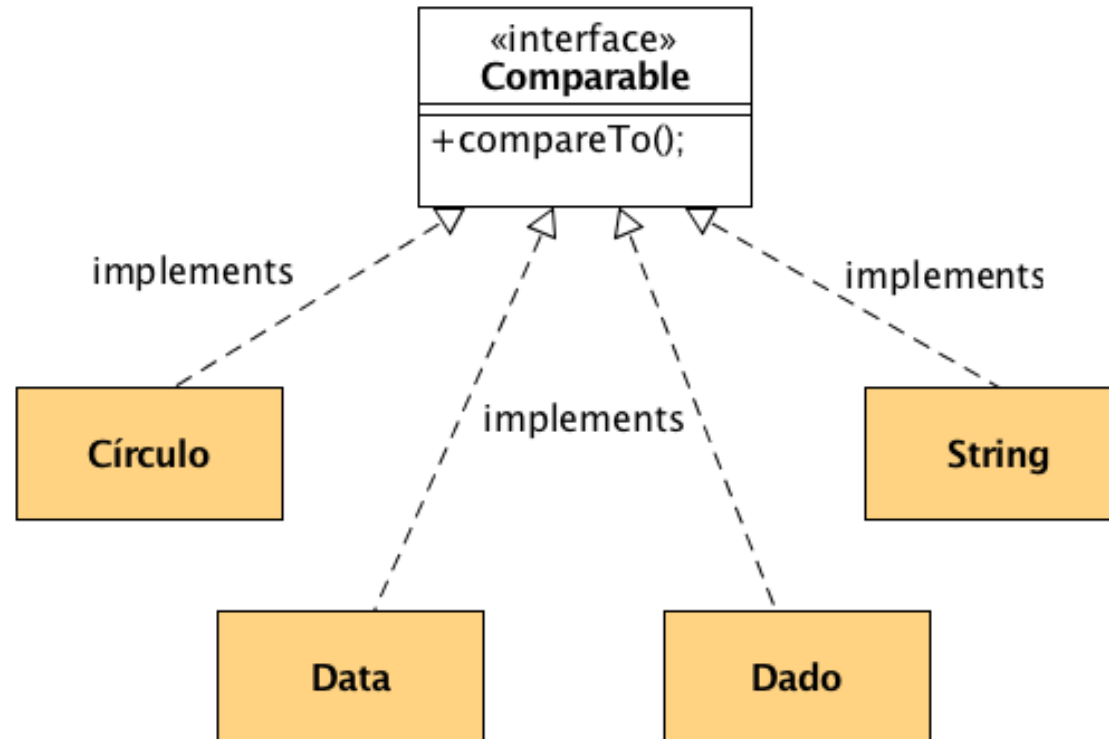
Collections – sorting

- ❖ How to compare elements in a collection?
 - custom types
 - objects that aren't directly comparable,
- ❖ Solution in the *Comparator* or *Comparable* interfaces
 - Classes / types follow comply with interface – Comparable interface
 - **Similar rationale to equals but with different classes**
<0 smaller, 0 equal, >1 larger
 - Interface to ask how they compare with other object

```
public interface Comparable<T> { // package java.lang;
    int compareTo(T other);
}
```

Questões?

- ❖ Porquê usar uma interface neste caso?
- ❖ Note que o método `int compareTo(T c)` retorna:
 - `<0` se `this < c`
 - `0` se `this == c`
 - `>0` se `this > c`



Comparable

```
public class Player implements Comparable<Player> {
    private int ranking;
    private String name;
    private int age;

    // constructor, getters, setters

    @Override
    public int compareTo(Player otherPlayer) {
        return Integer.compare(getRanking(), otherPlayer.getRanking());
    }
}
```

Compara rankings

```
public static void main(String[] args) {
    List<Player> footballTeam = new ArrayList<>();
    Player player1 = new Player(59, "John", 20);
    Player player2 = new Player(67, "Roger", 22);
    Player player3 = new Player(45, "Steven", 24);
    footballTeam.add(player1);
    footballTeam.add(player2);
    footballTeam.add(player3);

    System.out.println("Before Sorting : " + footballTeam);
    Collections.sort(footballTeam);
    System.out.println("After Sorting : " + footballTeam);
}
```

```
Before Sorting : [John, Roger, Steven]
After Sorting : [Steven, John, Roger]
```

Comparator

```
public class PlayerRankingComparator implements Comparator<Player> {  
  
    @Override  
    public int compare(Player firstPlayer, Player secondPlayer) {  
        return Integer.compare(firstPlayer.getRanking(), secondPlayer.getRanking());  
    }  
  
}
```

Compara rankings

```
public class PlayerAgeComparator implements Comparator<Player> {  
  
    @Override  
    public int compare(Player firstPlayer, Player secondPlayer) {  
        return Integer.compare(firstPlayer.getAge(), secondPlayer.getAge());  
    }  
  
}
```

Compara idades

Using this approach, we can override the natural ordering:

<https://www.baeldung.com/java-comparator-comparable>

comparator

```
public static void main(String[] args) {  
    List<Player> footballTeam = new ArrayList<>();  
    Player player1 = new Player(59, "John", 20);  
    Player player2 = new Player(67, "Roger", 22);  
    Player player3 = new Player(45, "Steven", 24);  
    footballTeam.add(player1);  
    footballTeam.add(player2);  
    footballTeam.add(player3);  
  
    System.out.println("Before Sorting : " + footballTeam);  
    Collections.sort(footballTeam);  
    System.out.println("After Sorting : " + footballTeam);  
}
```

```
PlayerAgeComparator playerComparator = new PlayerAgeComparator();  
Collections.sort(footballTeam, playerComparator);
```

```
Before Sorting : [John, Roger, Steven]  
After Sorting by age : [Roger, John, Steven]
```

```
PlayerRankingComparator playerComparator = new PlayerRankingComparator();  
Collections.sort(footballTeam, playerComparator);
```

```
Before Sorting : [John, Roger, Steven]  
After Sorting by ranking : [Steven, John, Roger]
```

Interface Comparable

```
public interface Comparable<T> { // package java.lang;
    int compareTo(T other);
}

public abstract class Shape implements Comparable<Shape> {
    public abstract double area( );
    public abstract double perimeter( );

    public int compareTo( Shape irhs ) {
        double res = area() - irhs.area();
        if (res > 0) return 1;
        else if (res < 0) return -1;
        else return 0;
    }
}
```

Interface Comparable

```
public class UtilCompare {

    // vamos discutir "<T extends Comparable<T>>" mais tarde
    public static <T extends Comparable<T>> findMax(T[] a) {
        int maxIndex = 0;
        for (int i = 1; i < a.length; i++)
            if (a[i] != null && a[i].compareTo(a[maxIndex]) > 0)
                maxIndex = i;
        return a[maxIndex];
    }

    public static <T extends Comparable<T>> void sortArray(T[] a) {
        // ...
    }
}
```

Interface Comparable

```
class FindMaxDemo {
    public static void main( String [ ] args ) {
        Figura[] sh1 = {
            new Circulo(1, 3, 1),          // x, y, raio
            new Quadrado(3, 4, 2),         // x, y, lado
            new Rectangulo(1, 1, 5, 6)     // x, y, lado1, lado2
        };
        String[] st1 = { "Joe", "Bob", "Bill", "Zeke" };
        System.out.println(UtilCompare.findMax(sh1));
        System.out.println(UtilCompare.findMax(st1));
    }
}
```

Rectangulo de Centro (1.0,1.0), altura 6.0, comprimento 5.0
Zeke

Why comparable?

Comparator and Comparable in Java

Last updated: March 17, 2024



Written by:
baeldung



Reviewed by:
Loredana Crusoveanu

<https://www.baeldung.com/java-comparator-comparable>

https://www.geeksforgeeks.org/comparable-interface-in-java-with-examples/?ref=header_outind

https://www.w3schools.com/java/java_advanced_sorting.asp

Sorting in collections

```
// Person class implementing Comparable
class Person implements Comparable<Person> {
    private String name;
    private int age;
    public Person(String name, int age) { (...) }

    public String getName() (...) }
    public int getAge() { (...) }

    @Override
    public int compareTo(Person other) {
        return Integer.compare(this.age, other.age); // Ascending order
    }

    public String toString() {
        return name + " (Age: " + age + ")";
    }
}
```


Sorting in collections

```
// Person class implementing Comparable
class Person implements Comparable<Person> {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public int compareTo(Person other) {
        // Sort the list (uses compareTo method in Person)
        Collections.sort(people);

        System.out.println("Sorted by age:");
        for (Person p : people) {
            System.out.println(p);
        }
    }
}
```

Sorting in collections

```
public class Main {  
    public static void main(String[] args) {  
        List<Person> people = new ArrayList<>();  
        people.add(new Person("Alice", 30));  
        people.add(new Person("Bob", 25));  
        people.add(new Person("Charlie", 35));  
  
        // Custom Comparator for sorting by name  
        Comparator<Person> nameComparator =  
            Comparator.comparing(Person:: getAge);  
  
        // Sort the list using the nameComparator  
        Collections.sort(people, nameComparator);  
  
        System.out.println("Sorted by age:");  
        for (Person p : people) {  
            System.out.println(p);  
        }  
    }  
}
```

Sorting in collections

```
public class Main {  
    public static void main(String[] args) {  
        List<Person> people = new ArrayList<>();  
        people.add(new Person("Alice", 30));  
        people.add(new Person("Bob", 25));  
        people.add(new Person("Charlie", 35));  
  
        // Custom Comparator for sorting by name  
        Comparator<Person> nameComparator =  
            Comparator.comparing(Person:: getName);  
  
        // Sort the list using the nameComparator  
        Collections.sort(people, nameComparator);  
  
        System.out.println("Sorted by name:");  
        for (Person p : people) {  
            System.out.println(p);  
        }  
    }  
}
```

Sorting in collections

```
public class Main {  
    public static void main(String[] args) {  
        List<Person> people = new ArrayList<>();  
        people.add(new Person("Alice", 30));  
        people.add(new Person("Bob", 25));  
        people.add(new Person("Charlie", 35));  
  
        // Custom Comparator that also uses Comparable-style comparison  
        Comparator<Person> nameComparator =  
            (p1, p2) -> p1.getName().compareTo(p2.getName());  
  
        // Sort by name using the custom Comparator  
        Collections.sort(people, nameComparator);  
  
        System.out.println("Sorted by name:");  
        for (Person p : people) {  
            System.out.println(p);  
        }  
    }  
}
```

Sorting in collections

```
public class Main {  
    public static void main(String[] args) {  
        List<Person> people = new ArrayList<>();  
        people.add(new Person("Alice", 30));  
        people.add(new Person("Bob", 25));  
        people.add(new Person("Charlie", 35));  
  
        // Custom Comparator that also uses Comparable-style comparison  
        Comparator<Person> ageComparator =  
            (p1, p2) -> p1.getAge() < p2.getAge ();  
  
        // Sort by name using the custom Comparator  
        Collections.sort(people, ageComparator);  
  
        System.out.println("Sorted by name:");  
        for (Person p : people) {  
            System.out.println(p);  
        }  
    }  
}
```

Sumário

- ❖ Polimorfismos
 - Static-binding
 - Dynamic-binding
- ❖ Classes abstratas
- ❖ Interface *Comparable*

The end